A Project Proposal Report on

Cipher++: A Multi-Algorithm Encryption and Decryption Toolkit

Submitted in Partial Fulfilment of the Requirements for

The Degree of Bachelor of Science in Computer Science

King's College Nepal

Submitted by:

Adeen Bajra Bajracharya, WU0259007

Date:

10/08/2026

King's College Nepal

Bijulibajar, Kathmandu

ABSTRACT

"Cipher++" is a menu-driven C++ application that allows users to encrypt and decrypt text using multiple classical cryptographic algorithms, including the Caesar Cipher, the Vigenère Cipher, and the XOR Cipher. Rather than building a separate program for every algorithm, Cipher++ brings them together into a single application where a user can pick an algorithm, enter a message and a key, and view the resulting output.

The primary purpose of the project is to demonstrate core Object-Oriented Programming (OOP) concepts rather than to advance cryptographic research. Every cipher is implemented as its own class that inherits from a common abstract `Cipher` class, which lets the application treat all ciphers uniformly through a shared interface while each class independently handles its own encryption and decryption logic.

The application supports encrypting and decrypting text, comparing the output produced by different algorithms on the same input, and generating keys through a dedicated `KeyGenerator` component. The project is built with a simple, menu-driven console interface and is intended to serve as a practical, undergraduate-level demonstration of encapsulation, inheritance, polymorphism, and abstraction in C++.

Keywords: Encryption, Decryption, Caesar Cipher, Vigenère Cipher, XOR Cipher, Object-Oriented Programming, C++

TABLE OF CONTENTS

LIST OF FIGURES

1. INTRODUCTION

Cryptography is one of the foundational topics in computer science, and learning it hands-on is one of the best ways to understand how information can be protected from unauthorized access. This project, Cipher++, is motivated by the goal of building a single, well-structured application that lets a user experiment with several classical encryption techniques side by side, while at the same time serving as a concrete, working example of Object-Oriented Programming design in C++. This chapter describes the problem being addressed, the motivation behind the project, its objectives, and its scope and limitations.

1.1 PROBLEM STATEMENT

Most introductory implementations of classical ciphers are written as small, standalone scripts a separate program for Caesar Cipher, another for Vigenère Cipher, another for XOR Cipher with little to no shared structure between them. This approach makes it difficult to compare algorithms directly, duplicates a large amount of code, and does not reflect how real-world software is organized.

Additionally, such scattered implementations rarely demonstrate good software design practices. They typically lack a common interface between algorithms, offer no easy way to extend the program with a new cipher, and provide no centralized way to generate or validate keys. This makes them poor learning tools for students who are trying to understand how OOP principles apply to real software.

1.2 PROJECT OBJECTIVES

To address the problems stated above, Cipher++ is proposed with the following objectives:

1. **Provide a single unified application for multiple classical encryption algorithms:** Users should be able to encrypt or decrypt text using the Caesar Cipher, Vigenère Cipher, or XOR Cipher from one menu-driven interface.
2. **Demonstrate core OOP principles in a practical setting:** The project is structured so that encapsulation, inheritance, polymorphism, and abstraction are directly visible in the class hierarchy and application design.
3. **Allow direct comparison between algorithms:** Users should be able to run the same message and key through multiple ciphers and compare the resulting outputs.
4. **Simplify key generation:** A dedicated `KeyGenerator` component will generate and validate keys for the selected cipher.
5. **Keep the application easy to extend:** The abstract `Cipher` base class is designed so that new algorithms can be added later as additional subclasses without modifying the rest of the application.

1.3 SCOPE AND LIMITATIONS

The scope of this project is as follows:

- The application targets students and self-learners who want a hands-on, practical introduction to classical cryptography and Object-Oriented Programming in C++.
- It supports three classical algorithms at launch Caesar Cipher, Vigenère Cipher, and XOR Cipher with the class structure designed to allow more algorithms to be added in the future.
- The application provides encryption, decryption, key generation, key validation, and comparison of results through a menu-driven console interface.

The limitations of this project are:

- Cipher++ implements classical, educational ciphers only; it is not intended for securing sensitive or production data.
- The application runs as a console-based command-line program; a graphical user interface is not part of the initial scope.
- Key strength and validation are limited to what each classical algorithm supports.

2. METHODOLOGY

2.1 SOFTWARE DEVELOPMENT LIFECYCLE

The incremental model of software development will be followed as the framework for this project. Development will be broken into smaller increments, each passing through Requirement Analysis, Design, Coding, and Testing and Evaluation, with every increment delivering a partial but usable version of the application.

In the first increment, the core product will be built: the abstract `Cipher` class, the `CaesarCipher` and `XORCipher` implementations, and a basic `MainScreen` menu that can encrypt and decrypt text. Subsequent increments will add the `VigenereCipher` class, the `KeyGenerator` component, and algorithm comparison.

2.1.1 REQUIREMENT ANALYSIS

This is the first step of each increment, where the required features are identified and clarified. The outcome of this phase is a short requirement note describing what the increment must deliver.

2.1.2 DESIGN

In this phase, the class structure for the increment is planned, including which classes are needed, how they relate to the abstract `Cipher` class, and what data each class is responsible for.

2.1.3 CODING

The functionality identified during design is implemented in C++. The outcome of this phase is working, compiled code implementing the increment's features.

2.1.4 TESTING AND EVALUATION

Each cipher class is tested individually with sample messages and keys to confirm that encryption followed by decryption reproduces the original text. The application as a whole is then tested to confirm that the menu, algorithm selection, key validation, and cipher operations work together correctly.

2.2 TOOLS TO BE USED

GIT AND GITHUB:
Git will be used as the version control system to track changes to the codebase, and GitHub will be used to host the project repository and manage revisions.

VS CODE:
Visual Studio Code will be used as the primary code editor for writing and debugging the C++ source files.

G++ / GCC:
The GNU C++ compiler will be used to compile and build the application.

2.3 TECHNOLOGIES TO BE USED

C++:
C++ is the primary programming language for this project. Its support for classes, inheritance, and polymorphism makes it well suited to demonstrating OOP concepts directly in the code.

Standard Template Library (STL):
STL components such as `std::string` and `std::vector` will be used for handling text, keys, and collections where required.

3. SYSTEM DESIGN

3.1 UML CLASS DIAGRAM

The UML class diagram illustrates the static structure of the Cipher++ system. It shows the major classes, their private attributes and public methods, and the relationships between them. The abstract `Cipher` class provides the common interface for encryption and decryption, while `CaesarCipher`, `VigenereCipher`, and `XORCipher` inherit from it. The diagram also shows the relationship of `MainScreen` with the cipher classes and the `KeyGenerator` class.

Figure 1: UML Class Diagram of Cipher++

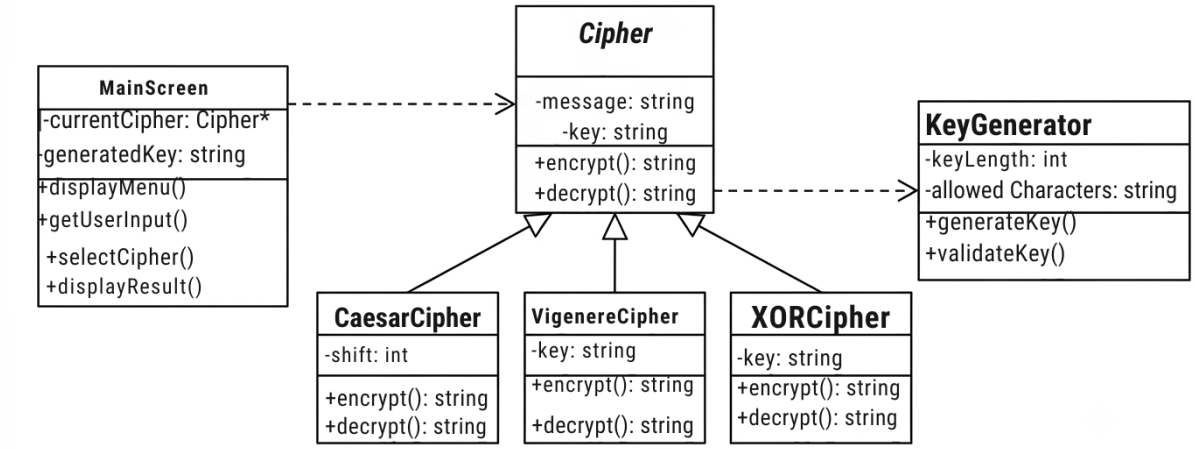

Figure 1: UML Class Diagram

3.2 UML SEQUENCE DIAGRAM

The UML sequence diagram illustrates the sequence of interactions that occur when a user performs an encryption or decryption operation. It shows how the user interacts with the `MainScreen`, how the key is validated through the `KeyGenerator`, and how the selected cipher performs the requested operation before returning the result to the user.

Figure 2: UML Sequence Diagram for Cipher++ Encryption

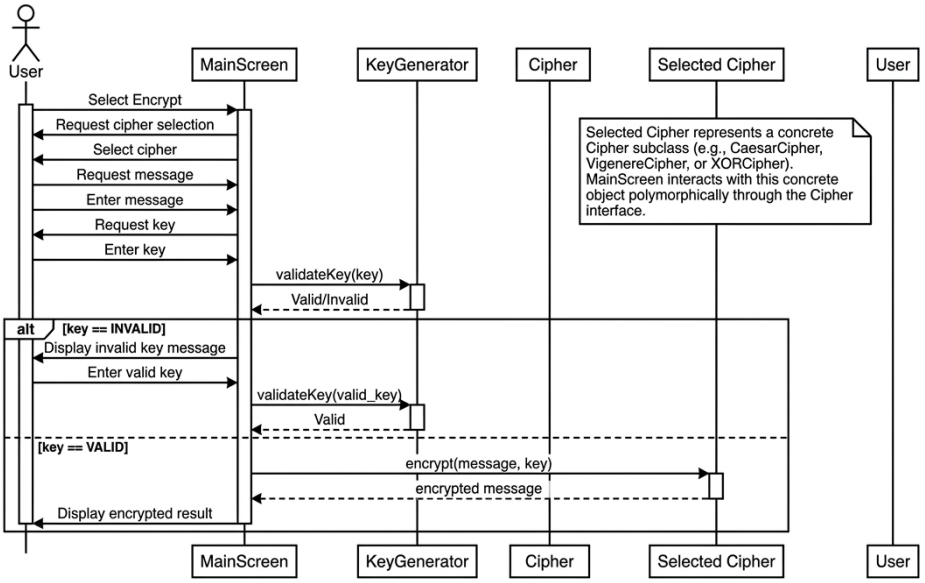

Figure 2: Sequence Diagram

3.3 UML USE CASE DIAGRAM

The UML use case diagram illustrates the functional requirements of Cipher++ from the perspective of the user. It shows the main operations available to the user, including encrypting and decrypting messages, selecting a cipher, entering or generating a key, validating the key, viewing results, and comparing the outputs of different encryption algorithms.

Figure 3: UML Use Case Diagram of Cipher++

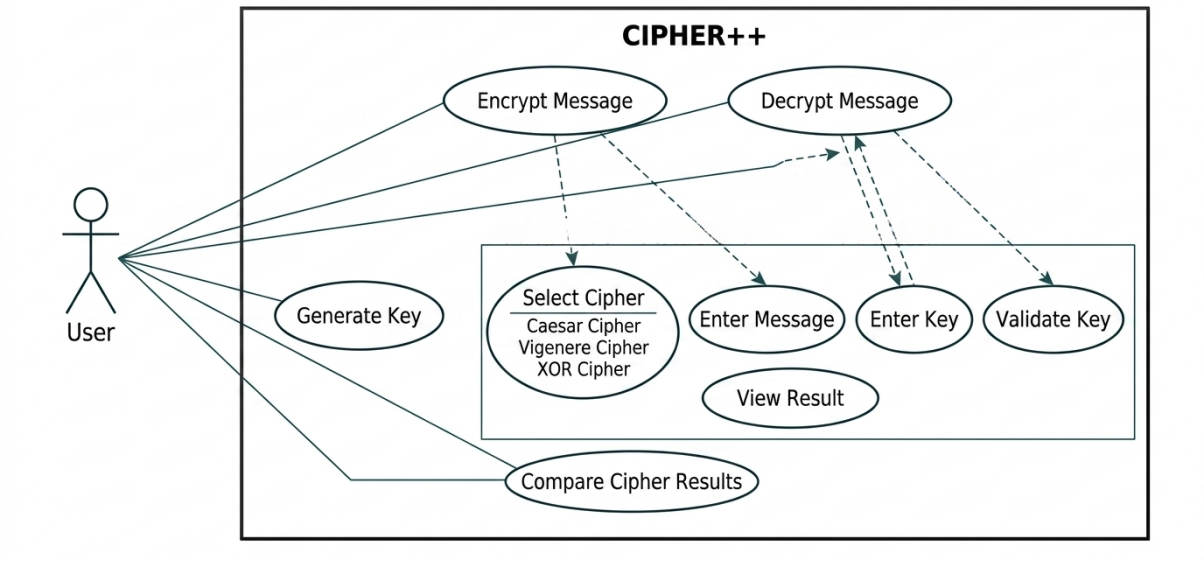

Figure 3: Use Case Diagram

3.4 OOP CONCEPTS DEMONSTRATED

Encapsulation:
Each class manages its own internal data and exposes only the functions needed by the rest of the application, keeping algorithm-specific details hidden inside each class.

Inheritance:
`CaesarCipher`, `VigenereCipher`, and `XORCipher` all inherit from the shared abstract `Cipher` class.

Polymorphism:
`MainScreen` can interact with a generic `Cipher` reference without needing to know which concrete cipher is being used.

Abstraction:
The `Cipher` class defines the common encryption and decryption interface while hiding the implementation details of each individual algorithm.

4. PROPOSED DELIVERABLES

The project will deliver a compiled, menu-driven C++ console application along with its documented source code. The application will allow a user to:

- Choose to encrypt or decrypt a message.
- Select from the Caesar Cipher, Vigenère Cipher, or XOR Cipher.
- Enter a message and a key, or generate a key automatically using `KeyGenerator`.
- Validate the entered key.
- View the resulting output on screen.
- Compare the output of different ciphers run on the same message.

BIBLIOGRAPHY

- Robert Sedgewick and Kevin Wayne, *Algorithms*, 4th Edition, Addison-Wesley, 2011.
- Bjarne Stroustrup, *The C++ Programming Language*, 4th Edition, Addison-Wesley, 2013.
- William Stallings, *Cryptography and Network Security: Principles and Practice*, 7th Edition, Pearson, 2017.